



Finding Shortest Path in Weighted Graph

Dijkstra's and Bellman-Ford Algorithm

Dr. Sumaiya Tabassum Nimi

Assistant Professor, Dept. of ECE — North South University

Introduction

Finding the shortest path between nodes in a weighted graph is one of the most important tasks in computer science, with applications in:

- GPS routing (Google Maps)
- Network packet routing
- Game AI movement
- Telecommunication and transport optimisation

Two key algorithms used for shortest-path problems are:

- **Dijkstra's Algorithm** — works with non-negative edge weights.
- **Bellman-Ford Algorithm** — works with negative weights and detects negative cycles.

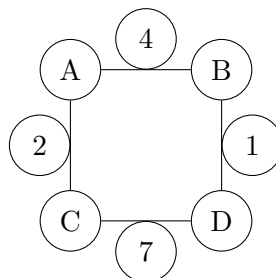
Both algorithms compute the shortest distance from a *single source* vertex to all other vertices.

Graph Representation

A weighted graph consists of:

- Vertices (nodes)
- Directed or undirected edges
- Edge weights $w(u, v)$

Below is a simple weighted graph drawn using TikZ.



Dijkstra's Algorithm

When to Use

Use Dijkstra's algorithm when:

- All edge weights are non-negative.
- You need fast computations.

Intuition

Dijkstra works like exploring ripples in a pond:

- Start at the source node.
- Expand outward, always picking the next nearest unvisited node.
- Update distances to neighbors.

Formal Algorithm

Algorithm 1 Dijkstra's Algorithm

```
1: Initialize distance array  $dist[v] = \infty$  for all vertices  $v$ 
2:  $dist[source] = 0$ 
3: Mark all vertices as unvisited
4: while there exists an unvisited vertex do
5:   Select unvisited vertex  $u$  with smallest  $dist[u]$ 
6:   Mark  $u$  as visited
7:   for each neighbor  $v$  of  $u$  do
8:     if  $dist[u] + w(u, v) < dist[v]$  then
9:        $dist[v] \leftarrow dist[u] + w(u, v)$ 
10:    end if
11:  end for
12: end while
```

Bellman–Ford Algorithm

The Bellman–Ford algorithm is a shortest path algorithm that works even when edge weights can be negative. To understand *why* it works, it helps to think about the problem in terms of “relaxing” edges repeatedly.

Key Intuition

Suppose we want to find the shortest path from a source node s to every other node in a graph. Any path from s to a node v can contain at most $(n - 1)$ edges in a graph with n vertices (because a simple path cannot repeat a vertex). Therefore:

- After one relaxation of all edges, we correctly compute the shortest paths that use *one* edge.
- After two relaxations of all edges, we correctly compute the shortest paths that use *two* edges.
- ...
- After $(n - 1)$ relaxations, we correctly compute the shortest paths that use *up to* $(n - 1)$ edges.

This is the essential idea: each pass of edge relaxation increases the maximum number of edges that the algorithm can “see” when constructing shortest paths.

What is Relaxation?

Relaxation is a simple update rule:

If going from u to v through an edge (u, v) gives a shorter distance than what we currently know for v , then update the distance for v .

Mathematically:

if $dist[v] > dist[u] + w(u, v)$ then update $dist[v] := dist[u] + w(u, v)$.

Each relaxation brings the algorithm closer to the true shortest path.

Why $(n - 1)$ Iterations?

A shortest path can have at most $(n - 1)$ edges. Therefore, after performing full relaxations $(n - 1)$ times, all shortest paths must have been discovered. If there is still an improvement possible after the $(n - 1)$ th iteration, this must be because the graph contains a *negative cycle*. Bellman–Ford can detect such cycles by checking for any further improvement.

Example of the Intuition

Imagine distances spreading outward from the source like ripples in water. Each relaxation round expands these ripples one step further, allowing shortest-path information to move through one more edge. After enough rounds, the ripples have passed through all paths that could matter.

Summary

- Bellman–Ford relies on repeated relaxation of all edges.
- Each iteration allows paths with one more edge to be considered.
- After $(n - 1)$ rounds, all possible simple paths have been explored.
- If improvements are still possible, the graph contains a negative cycle.

The algorithm is powerful because it works even when some edges have negative weights, as long as no negative cycle is reachable from the source.

Formal Algorithm

Algorithm 2 Bellman–Ford Algorithm

```
1: Initialize  $dist[v] = \infty$  for all vertices
2:  $dist[source] = 0$ 
3: for  $i = 1$  to  $|V| - 1$  do
4:   for each edge  $(u, v)$  with weight  $w(u, v)$  do
5:     if  $dist[u] + w(u, v) < dist[v]$  then
6:        $dist[v] \leftarrow dist[u] + w(u, v)$ 
7:     end if
8:   end for
9: end for
10: for each edge  $(u, v)$  do
11:   if  $dist[u] + w(u, v) < dist[v]$  then
12:     Report: Negative cycle exists!
13:   end if
14: end for
```

Key Differences

Feature	Dijkstra	Bellman–Ford
Negative weights	Not allowed	Allowed
Negative cycles	Cannot detect	Detects negative cycles
Time Complexity	$O(V^2)$ or $O((V + E) \log V)$	$O(VE)$

Summary

- Use **Dijkstra** for fast shortest paths with non-negative weights.
- Use **Bellman–Ford** when negative weights or cycles may exist.
- Both algorithms iteratively relax edges, but Dijkstra uses a greedy strategy while Bellman–Ford uses dynamic programming.

Worked Example: Dijkstra’s and Bellman–Ford on the Same Graph

In this example, we will run **both** Dijkstra’s algorithm and Bellman–Ford on the *same* weighted graph, starting from source vertex A . All edge weights are non-negative, so both algorithms will give the same shortest path distances.

Graph Description

We have a directed graph with vertices

$$V = \{A, B, C, D, E, F\}$$

and the following weighted edges:

From	To	Weight
A	B	4
A	C	2
B	C	5
B	D	10
C	E	3
E	D	4
D	F	11
E	F	5

We want to find the shortest path distances from A to all other vertices.

Source: A

We denote $d(v)$ as the current best known distance from A to v .

Initially (for both algorithms):

$$d(A) = 0, \quad d(B) = d(C) = d(D) = d(E) = d(F) = \infty$$

and all predecessors are *undefined*.

Part 1: Dijkstra's Algorithm (from A)

Initialization.

$$d(A) = 0, \quad d(B) = d(C) = d(D) = d(E) = d(F) = \infty.$$

The set of visited (finalized) vertices is $S = \emptyset$ at the beginning.

We always pick the **unvisited vertex with minimum $d(\cdot)$** next.

Step 1. Pick vertex with smallest d : clearly A ($d(A) = 0$).

Add A to S :

$$S = \{A\}.$$

Relax all edges leaving A :

- Edge $A \rightarrow B$ with weight 4:

$$d(B) > d(A) + 4 \Rightarrow \infty > 0 + 4 \Rightarrow d(B) = 4, \text{ pred}(B) = A.$$

- Edge $A \rightarrow C$ with weight 2:

$$d(C) > d(A) + 2 \Rightarrow \infty > 0 + 2 \Rightarrow d(C) = 2, \text{ pred}(C) = A.$$

Distances after Step 1:

Vertex	$d(\cdot)$	Predecessor
A	0	–
B	4	A
C	2	A
D	∞	–
E	∞	–
F	∞	–

Step 2. Among unvisited vertices $\{B, C, D, E, F\}$, the smallest d is $d(C) = 2$.

Add C to S :

$$S = \{A, C\}.$$

Relax edges leaving C :

- Edge $C \rightarrow E$ with weight 3:

$$d(E) > d(C) + 3 \Rightarrow \infty > 2 + 3 \Rightarrow d(E) = 5, \text{ pred}(E) = C.$$

Distances after Step 2:

Vertex	$d(\cdot)$	Predecessor
A	0	–
B	4	A
C	2	A
D	∞	–
E	5	C
F	∞	–

Step 3. Among unvisited vertices $\{B, D, E, F\}$, the smallest d is $d(B) = 4$.

Add B to S :

$$S = \{A, C, B\}.$$

Relax edges leaving B :

- Edge $B \rightarrow C$ has weight 5:

$$d(C) \leq d(B) + 5 \Rightarrow 2 \leq 4 + 5 \Rightarrow \text{no update.}$$

- Edge $B \rightarrow D$ has weight 10:

$$d(D) > d(B) + 10 \Rightarrow \infty > 4 + 10 \Rightarrow d(D) = 14, \text{ pred}(D) = B.$$

Distances after Step 3:

Vertex	$d(\cdot)$	Predecessor
A	0	–
B	4	A
C	2	A
D	14	B
E	5	C
F	∞	–

Step 4. Among unvisited vertices $\{D, E, F\}$, the smallest d is $d(E) = 5$.

Add E to S :

$$S = \{A, C, B, E\}.$$

Relax edges leaving E :

- Edge $E \rightarrow D$ with weight 4:

$$d(D) > d(E) + 4 \Rightarrow 14 > 5 + 4 \Rightarrow d(D) = 9, \text{ pred}(D) = E.$$

- Edge $E \rightarrow F$ with weight 5:

$$d(F) > d(E) + 5 \Rightarrow \infty > 5 + 5 \Rightarrow d(F) = 10, \text{ pred}(F) = E.$$

Distances after Step 4:

Vertex	$d(\cdot)$	Predecessor
A	0	–
B	4	A
C	2	A
D	9	E
E	5	C
F	10	E

Step 5. Among unvisited vertices $\{D, F\}$, the smallest d is $d(D) = 9$.

Add D to S :

$$S = \{A, C, B, E, D\}.$$

Relax edges leaving D :

- Edge $D \rightarrow F$ with weight 11 gives

$$d(F) \leq d(D) + 11 \Rightarrow 10 \leq 9 + 11 \Rightarrow \text{no update.}$$

Distances after Step 5:

Vertex	$d(\cdot)$	Predecessor
A	0	–
B	4	A
C	2	A
D	9	E
E	5	C
F	10	E

Step 6. The only remaining vertex is F .

Add F to S :

$$S = \{A, C, B, E, D, F\}.$$

There are no more updates. The algorithm terminates.

Final shortest-path distances from A :

$$d(A) = 0, \quad d(B) = 4, \quad d(C) = 2, \quad d(E) = 5, \quad d(D) = 9, \quad d(F) = 10.$$

Part 2: Bellman–Ford Algorithm (from A)

Bellman–Ford initializes distances in the same way, then repeatedly relaxes *all* edges $|V| - 1$ times (here $6 - 1 = 5$ times).

Initialization (Iteration 0).

$$d(A) = 0, \quad d(B) = d(C) = d(D) = d(E) = d(F) = \infty.$$

We show a summary of the updates per iteration.

Iteration 1. Relax all edges in any order:

- $A \rightarrow B$ (4): $d(B)$ becomes 4.
- $A \rightarrow C$ (2): $d(C)$ becomes 2.
- $B \rightarrow C$ (5): $d(C)$ compared with $4 + 5 = 9$ (no change).
- $B \rightarrow D$ (10): $d(D)$ becomes 14.
- $C \rightarrow E$ (3): $d(E)$ becomes 5.
- $E \rightarrow D$ (4): $d(D)$ becomes 9 (since $5 + 4 = 9 < 14$).
- $D \rightarrow F$ (11): $d(F)$ becomes 20.
- $E \rightarrow F$ (5): $d(F)$ becomes 10 (since $5 + 5 = 10 < 20$).

Distances after Iteration 1:

$$d(A) = 0, d(B) = 4, d(C) = 2, d(E) = 5, d(D) = 9, d(F) = 10.$$

Iterations 2, 3, 4, 5.

We again relax all edges in each iteration. However, in this example, *no further changes* occur after Iteration 1, because the shortest paths have already been found.

So at each subsequent iteration, $d(\cdot)$ remains:

$$d(A) = 0, d(B) = 4, d(C) = 2, d(E) = 5, d(D) = 9, d(F) = 10.$$

Comparison and Intuition

- Both Dijkstra's and Bellman–Ford eventually compute the same shortest-path distances from A .
- Dijkstra's uses a greedy selection of the next closest vertex and runs faster on non-negative graphs.
- Bellman–Ford does repeated relaxation over all edges and is slower, but it can also handle negative edge weights (as long as there are no negative cycles).

In this example, the graph has only non-negative weights, so Dijkstra's is more efficient, but Bellman–Ford still converges to the same answer.